

Absolutely, buddy. Let's convert the idea into a **business-driven IBM-style project**, not just a coding project.

Final Project Idea

AI-Powered Cloud Incident Management and Observability Platform

This project is a **web-based platform for engineering teams** to monitor cloud applications, detect service failures, analyze logs, and generate AI-powered incident summaries.

IBM's software direction strongly connects with this idea because IBM focuses heavily on **enterprise AI, hybrid cloud, containerized applications, and observability/AIOps**. IBM watsonx.ai is positioned for building and scaling enterprise AI applications, Red Hat OpenShift on IBM Cloud supports hybrid/multicloud container deployment, and IBM Instana focuses on AI-powered observability for DevOps and engineering teams. ([IBM](#))

1. Targeted Audience

This project is mainly for:

Audience	Why They Need It
DevOps Engineers	Monitor deployments, containers, logs, and failures
Backend Engineers	Debug API failures, database issues, and service crashes
SRE Teams	Reduce downtime and respond to incidents faster
Engineering Managers	Track system reliability and incident history
Cloud Administrators	Monitor Kubernetes/cloud service health
Enterprise Support Teams	Understand customer-facing outages faster

This is not a normal user-facing app like food delivery or e-commerce. It is an **enterprise engineering platform**, which matches IBM better.

2. Business Problem Being Solved

Real Business Problem

Modern companies run many services:

- auth-service
- payment-service
- user-service
- order-service
- notification-service
- database-service

When something breaks, engineers usually need to manually check:

- logs
- metrics
- database status
- API errors
- deployment history
- container status
- recent code changes

This takes time. During that delay, the business may lose:

- revenue
- customer trust
- service uptime
- engineering productivity
- support efficiency

Strong Business Motive

The project solves this problem:

Engineering teams waste too much time identifying the root cause of production incidents because logs, metrics, deployment data, and service health are scattered across multiple systems.

Your platform solves it by giving teams a **single AI-powered dashboard** that detects incidents, summarizes root causes, and recommends debugging steps.

3. Real-Time Use Case

Example Scenario

A company has an online payment system.

Suddenly, users cannot complete payments.

The backend logs show:

```
payment-service ERROR: Database connection timeout  
payment-service ERROR: Failed to process transaction  
payment-service ERROR: Connection pool exhausted
```

Normally, engineers may take time to manually inspect logs.

Your system will automatically:

1. Detect repeated ERROR logs from payment-service
 2. Create a Critical incident
 3. Show affected service on dashboard
 4. Generate AI root cause summary
 5. Suggest debugging steps
 6. Notify the engineering team
 7. Track incident status until resolved
-

AI Output Example

Incident Summary:

The payment-service is experiencing repeated database connection timeouts.

Possible Root Cause:

The PostgreSQL connection pool may be exhausted or the database container may be unhealthy.

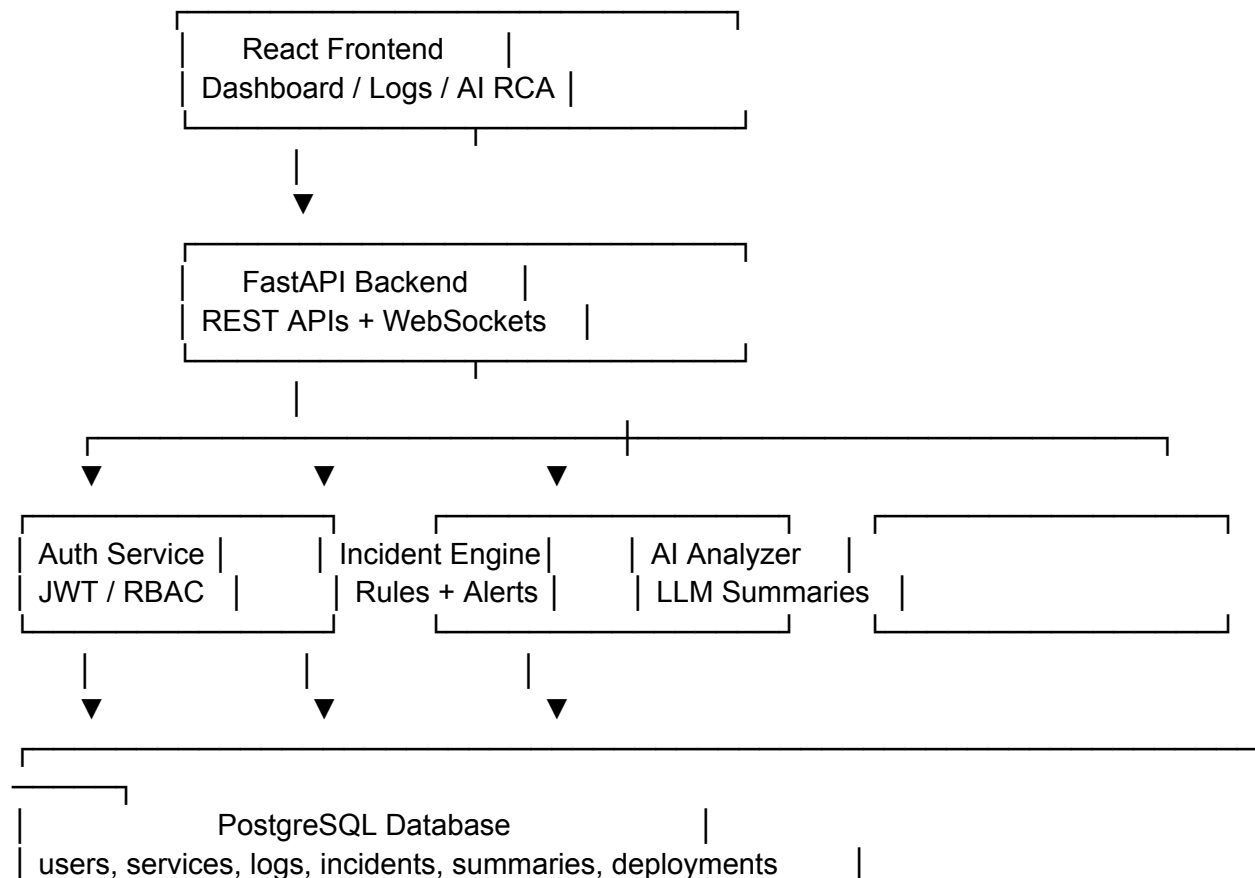
Recommended Debugging Steps:

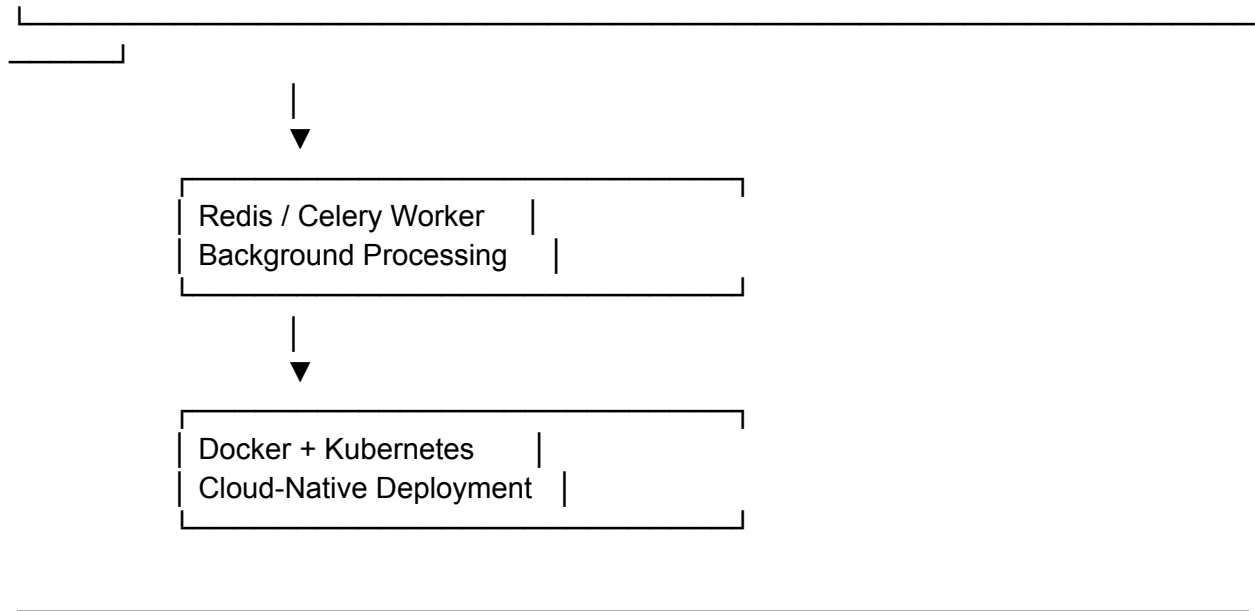
1. Check PostgreSQL container health.
2. Verify database connection pool settings.
3. Review recent deployments.
4. Check service environment variables.
5. Restart payment-service if the database is healthy.

This is a strong real-world enterprise use case.

4. Final Project Architecture

High-Level Architecture





5. Website Pages

1. Landing Page

Purpose: Explain the product.

Sections:

Hero section
Problem statement
Platform benefits
How it works
Technology stack
Demo button

Example headline:

AI-powered incident management for cloud-native engineering teams.

2. Login / Signup Page

Purpose: Secure access.

Features:

User registration

Login

JWT authentication

Role-based access

Admin / Engineer roles

3. Main Dashboard

Purpose: Show system health quickly.

Cards:

Total services

Healthy services

Warning services

Critical incidents

Average response time

Recent deployments

Open incidents

Resolved incidents

Dashboard example:

Services Monitored: 8

Healthy: 5

Warning: 2

Critical: 1

Open Incidents: 3

4. Service Health Page

Purpose: Track each microservice.

Features:

Service name

Status

Uptime
Error count
Average latency
Last deployment
Recent logs
Container status

Example:

payment-service	Critical
auth-service	Healthy
order-service	Warning
notification-service	Healthy

5. Logs Page

Purpose: Search and filter logs.

Features:

- Filter by service
- Filter by log level
- Filter by timestamp
- Filter by trace ID
- Search error messages
- View raw logs

Log levels:

- INFO
- WARNING
- ERROR
- CRITICAL

6. Incidents Page

Purpose: Track system failures.

Features:

Incident ID
Affected service
Severity
Status
Created time
Assigned engineer
Root cause
Resolution notes

Incident statuses:

Open
Investigating
Resolved
Closed

Severity levels:

Low
Medium
High
Critical

7. AI Root Cause Analysis Page

Purpose: Show AI-generated explanation.

Features:

Incident summary
Possible root cause
Suggested debugging steps
Related logs
Confidence score
Engineer feedback

This is the most important page for IBM alignment because it connects **AI + debugging + cloud-native software**.

8. Deployment History Page

Purpose: Connect failures to recent releases.

Features:

Deployment version

Service name

Commit ID

Deployment time

Status

Rollback button

Related incidents

Example:

payment-service v1.2.4 deployed at 10:02 AM

incident created at 10:08 AM

This helps engineers identify whether a new deployment caused the issue.

9. Notifications Page

Purpose: Alert teams when incidents happen.

Features:

Email alerts

Slack-style webhook alerts

Incident assignment

Critical alert history

Notification preferences

For resume, you can implement email alerts first.

10. Admin Settings Page

Purpose: Manage services and thresholds.

Features:

- Add service
- Update service owner
- Set alert thresholds
- Manage users
- Manage roles
- Configure AI provider
- Configure database/API keys

6. Complete Feature List

Core MVP Features

These are enough for a strong resume project:

- User authentication
- React dashboard
- Service health monitoring
- Log ingestion API
- Log filtering/search
- Incident detection rules
- Incident creation
- AI root cause summary
- PostgreSQL persistence
- Docker Compose setup
- REST APIs
- Unit testing
- GitHub Actions CI pipeline

Advanced Features

Add these to make it IBM-level:

Kubernetes deployment files
WebSocket real-time incident updates
Role-based access control
Prometheus metrics
Grafana dashboard
Redis/Celery background workers
Deployment history tracking
AI confidence score
Engineer feedback on AI summaries
OpenShift-compatible deployment documentation

IBM Cloud Monitoring and Instana focus on cloud-native/container observability and real-time application insight, so adding observability-style features like logs, metrics, traces, and service health makes the project especially relevant. ([IBM](#))

7. Backend Services

Recommended Backend Modules

auth-service
service-registry-module
log-ingestion-module
incident-detection-module
ai-analysis-module
notification-module
deployment-tracking-module

You can keep them inside one FastAPI backend for MVP, but structure the folders like microservices.

8. Database Design

Main Tables

users

services
logs
incidents
incident_summaries
deployments
notifications

services

id
name
description
owner
status
environment
created_at

logs

id
service_id
level
message
trace_id
timestamp
created_at

incidents

id
service_id
severity
status
title
description
created_at
resolved_at

incident_summaries

id
incident_id

ai_summary
possible_root_cause
recommended_steps
confidence_score
created_at

deployments

id
service_id
version
commit_id
status
deployed_at

9. API Architecture

Main REST APIs

POST /auth/register
POST /auth/login

GET /services
POST /services
GET /services/{id}
PATCH /services/{id}

POST /logs
GET /logs
GET /logs/{service_id}

GET /incidents
POST /incidents
GET /incidents/{id}
PATCH /incidents/{id}

POST /ai/analyze/{incident_id}
GET /ai/summary/{incident_id}

GET /deployments

POST /deployments

GET /dashboard/summary

GET /health

Real-Time API

/ws/incidents

Use WebSocket to update the dashboard when a new incident is created.

10. Suggested Tech Stack

Layer	Technology
Frontend	React, TypeScript, Tailwind CSS
Backend	FastAPI, Python
Database	PostgreSQL
AI Layer	OpenAI / Hugging Face / IBM watsonx-style integration
Background Jobs	Celery
Cache / Queue	Redis
Containerization	Docker
Orchestration	Kubernetes
CI/CD	GitHub Actions
Testing	Pytest, React Testing Library
Monitoring	Prometheus, Grafana
Version Control	Git, GitHub

11. Website User Flow

Flow 1: Engineer Monitors Services

Login

- Dashboard
- View service health
- Click unhealthy service
- View logs
- Check related incidents
- Read AI root cause summary
- Update incident status

Flow 2: System Detects Incident

Logs arrive

- Incident engine checks error threshold
- Critical incident is created
- AI analyzer generates summary
- Dashboard updates in real time
- Engineer receives notification

Flow 3: Admin Adds New Service

Login as Admin

- Go to Settings
 - Add service
 - Set alert threshold
 - Save service
 - Start monitoring logs
-

12. Resume Positioning

Use this project title:

AI-Powered Cloud Incident Management Platform

Resume bullets:

```
\resumeProjectHeading
{\textbf{AI-Powered Cloud Incident Management Platform} }{\emph{React, FastAPI,
PostgreSQL, Docker, Kubernetes, LLMs}}
\resumeItemListStart
  \resumeItem{Designed and developed an AI-powered observability platform that monitors
cloud-native services, detects production incidents, and generates root cause summaries using
LLMs.}
  \resumeItem{Built FastAPI REST APIs for service registry, log ingestion, incident tracking,
deployment history, and AI-generated debugging recommendations.}
  \resumeItem{Developed a React dashboard for real-time service health monitoring, incident
severity classification, log search, and engineering workflow management.}
  \resumeItem{Containerized frontend, backend, PostgreSQL, Redis, and worker services using
Docker and structured Kubernetes manifests for cloud-ready deployment.}
  \resumeItem{Implemented Pytest-based API tests and GitHub Actions CI workflow to improve
code quality, reliability, and deployment readiness.}
\resumeItemListEnd
```

13. Why This Project Is Strong for IBM

This project directly shows:

- AI-enabled software
- Cloud-native development
- Backend APIs
- React frontend
- Docker
- Kubernetes
- REST APIs
- Databases
- CI/CD
- Debugging
- Testing
- Agile-style modular design
- Enterprise problem-solving

That is exactly the story you want IBM to see.

The final message your resume should communicate is:

I can build modern enterprise software that combines AI, cloud-native architecture, APIs, frontend dashboards, containers, testing, and real business problem-solving.